Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

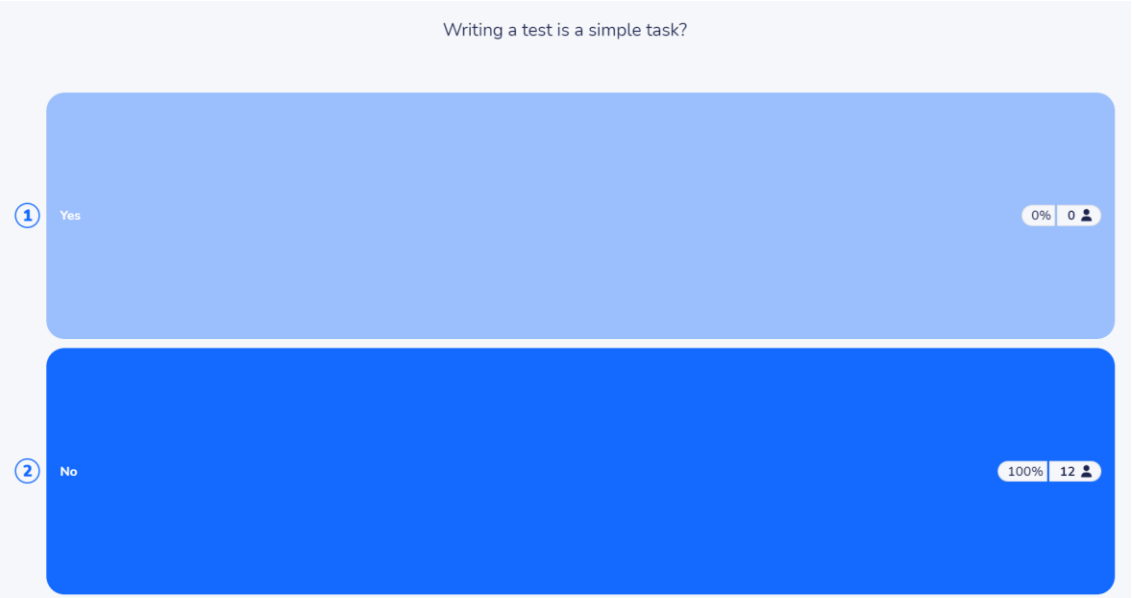
matteo.zanioli@unive.it

Your opinion

Writing a test is a simple task?

Ca' Foscari University of Venice

Your opinion



Overview

- It appears an easy task, but experience show me not (remember the halting problem)
- User behavior could be unpredictable (or better, in general users have the fantastic skill to surprise you)
- You have to consider always the context: the sensor of Arianne detect that the rocket was in a position that require to stop the engine and it fault

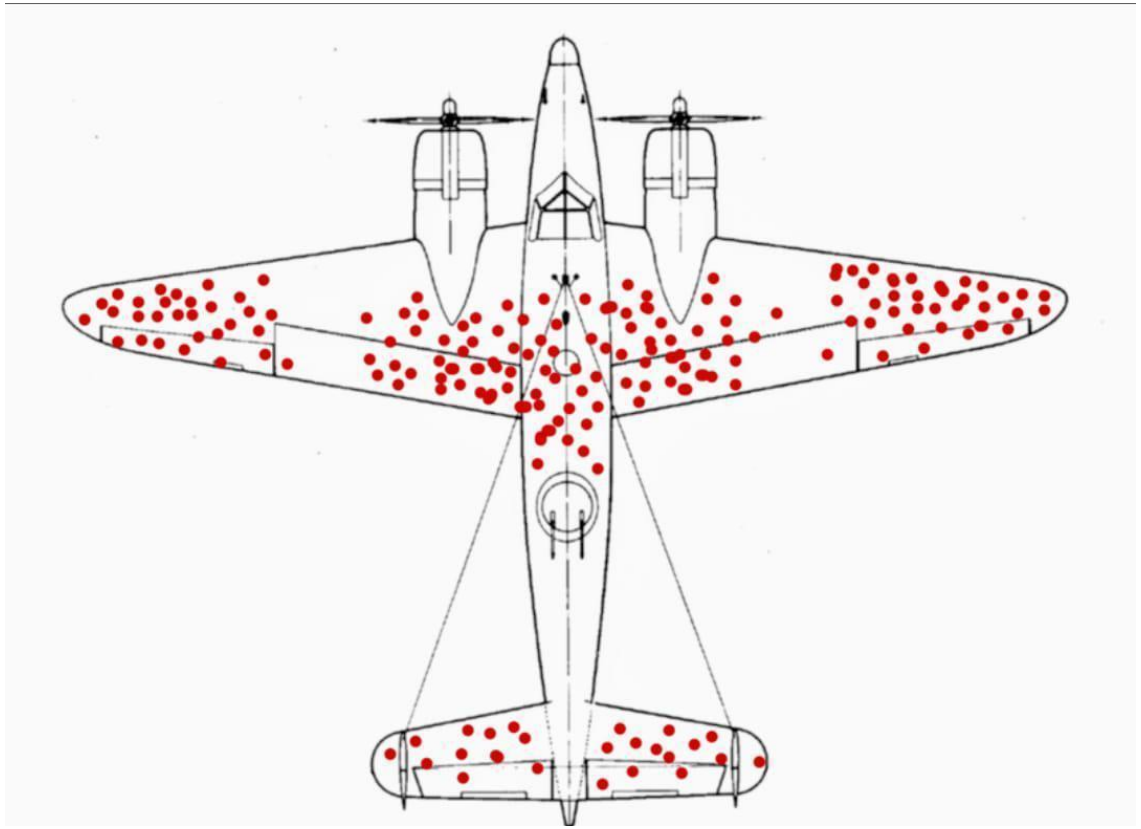
Ca' Foscari University of Venice

Overview

Ca' Foscari University of Venice

Look the thing from different perspective

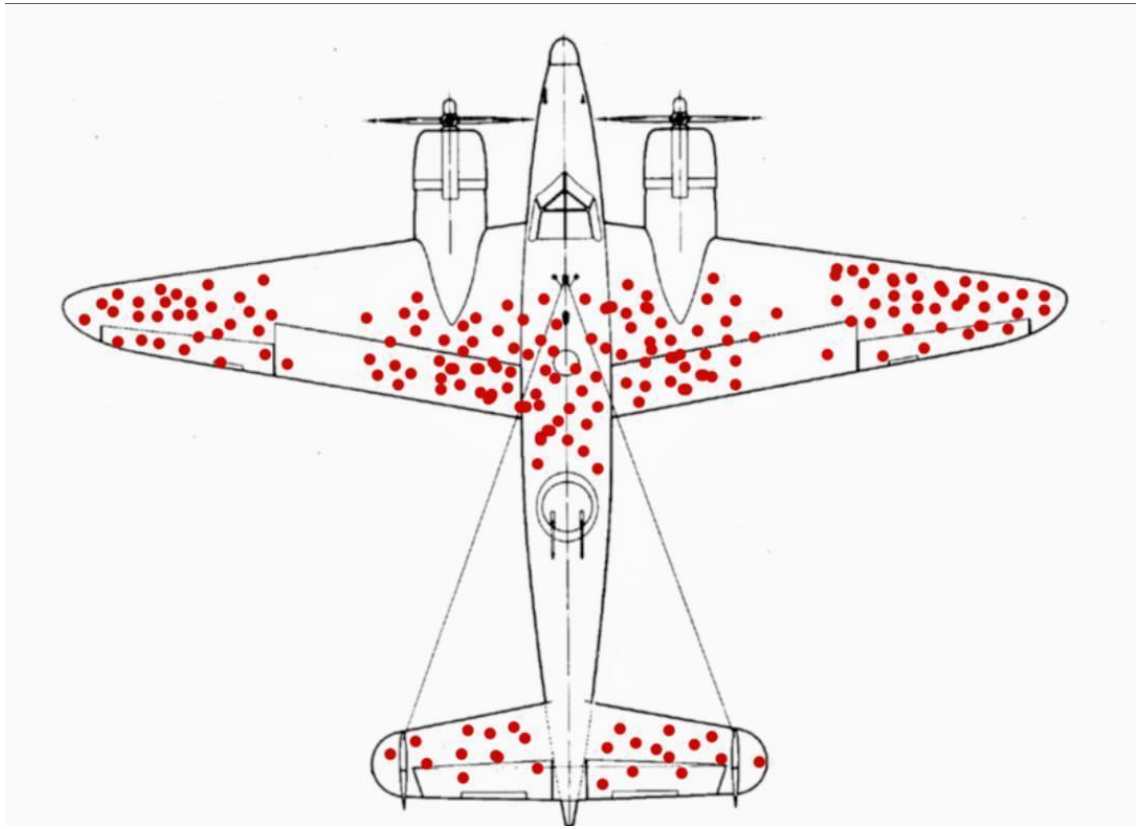
- Abraham Wald (1902-1950)
- mathematician who made a remarkable contribution during World War II by using statistical thinking to solve a critical problem in military aviation
- Planes returning from missions were often riddled with bullet holes, and military engineers wanted to figure out where to reinforce the planes with extra armor to minimize losses



Ca' Foscari University of Venice

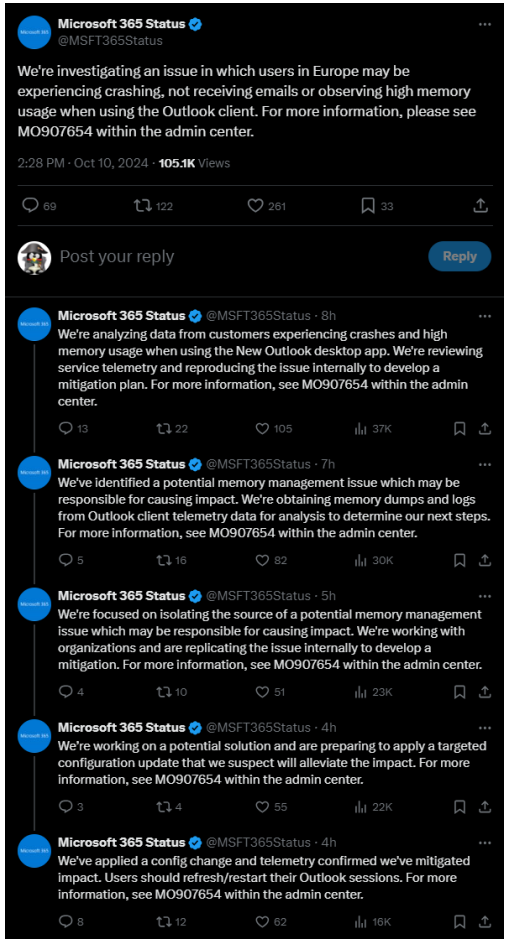
Look the thing from different perspective

- The obvious solution seemed to be to look at the planes that had returned from combat, analyze where they had sustained the most damage, and reinforce those areas.
- The planes they were examining were the ones that **made it back**. In other words, these were the planes that had sustained damage but had **survived**. The bullet holes on the returning planes showed where planes could be hit **and still fly**.
- He recommended reinforcing the areas that had no bullet holes on the returning planes
- This concept is now famously referred to as "survivorship bias." Wald's analysis helped the Allies save countless lives and aircraft by optimizing where armor was applied



Ca' Foscari University of Venice

MS Outlook bug



- Bug 10/10/2024
- Not only Europe, it seems all around the world
- They are talking about a bug!

- Not so easy test everything.
- Screenshot taken at 23:54 of 10/10/2024

A big common constraints: time

- You do not have the time to test all possibility (e.g., if you have implemented a calculator you could spent all your life without have the confirmation that it works with all numbers – it will never pass all the tests because you have infinite cases to test)
- You have to find the right model to test some specific properties (again, remember the halting problem)

A first guideline

- Happy path (Positive test cases)
  - These tests verify the software works as expected under ideal condition.
- Negative test cases
  - These tests check how the software handles invalid, incorrect, or unexpected inputs.
- Boundary conditions
  - Test edge cases, such as the minimum and maximum input values
- Error Handling and recovery tests
  - Validate how the software handles unexpected failures, such as network outages, server crashes, or corrupted data
- Concurrency tests
  - Test how the system behaves when multiple operations happen simultaneously
- Performance tests
  - Evaluate how the system performs under load or stress; this includes testing for response time, throughput, and scalability under different conditions

A first guideline

- Security tests
  - check for vulnerabilities like SQL injection, cross-site scripting, authentication flaws and other security threats
- Usablity tests
  - these tests assess how easy it is for users to interact with the software
- Compatibility tests
  - Ensure the software works across different environments (browsers, operating systems, device, ...)
- Integration tests
  - Verify that different modules or services within the application work well together
- End-to-end tests
  - These simulate real-world user scenarios to ensure the entire application workflow functions from start to finish
- Localization and internationalization tests
  - for global applications, test how the software adapts to different languages, locales and cultural formats (Date, numbers, currencies, ...)

Some examples

```python
def is_valid_age(age: int) -> bool:
    if 0 <= age <= 120:
        return True
    return False
```

simple function that checks if a given integer is a valid age (between 0 and 120)

Some examples

```python
class TestIsValidAge(unittest.TestCase):

    # Happy Path
    def test_happy_path(self):
        self.assertTrue(is_valid_age(25))  # A typical valid age
        self.assertTrue(is_valid_age(0))   # Edge of lower bound
        self.assertTrue(is_valid_age(120)) # Edge of upper bound

    # Negative Test Cases
    def test_negative_cases(self):
        self.assertFalse(is_valid_age(-1))  # Negative age is invalid
        self.assertFalse(is_valid_age(121)) # Age above 120 is invalid
        self.assertFalse(is_valid_age(999)) # Unrealistically large age is invalid
        self.assertFalse(is_valid_age("twenty")) # Non-integer input

    # Boundary Conditions
    def test_boundary_conditions(self):
        self.assertTrue(is_valid_age(0))    # Lower boundary case
        self.assertTrue(is_valid_age(120))  # Upper boundary case
        self.assertFalse(is_valid_age(-1))  # Just below lower boundary
        self.assertFalse(is_valid_age(121)) # Just above upper boundary
```

- happy path – tests with values that are clearly within the valid range
- negative test cases – tests with invalid inputs (negative numbers or non-integer)
- boundary conditions – tests with lower boundary (0), upper boundary (120) and just outside these boundaries (-1 and 121)

ATM example - statement

- Send me an e-mail to add you on my github repo: you will be able to see all the code of the following example.
- I create a Java application which simulate an ATM and I create a set of tests to show you how they work.
- It is a easy example, but useful to show you different cases.

Ca' Foscari University of Venice

ATM example – error handling

```java
/**
 * Constructs a new BankAccount with the specified initial balance.
 *
 * @param initialBalance the initial balance for the account
 * @throws IllegalArgumentException if the initial balance is negative
 */
public BankAccount(double initialBalance) {
    if (initialBalance < 0) {
        throw new IllegalArgumentException("Initial balance cannot be negative");
    }
    this.balance = initialBalance;
}
```

```java
/**
 * Tests that the constructor throws exception for negative initial balance.
 */
private static void testConstructorWithNegativeBalance() {
    testsTotal++;
    try {
        new BankAccount(-50.0);
        System.out.println("❌ testConstructorWithNegativeBalance failed: Should have thrown IllegalArgumentException");
    } catch (IllegalArgumentException e) {
        testsPassed++;
        System.out.println("✅ testConstructorWithNegativeBalance passed");
    } catch (Exception e) {
        System.out.println("❌ testConstructorWithNegativeBalance failed: Unexpected exception: " + e.getMessage());
    }
}
```

- Error handling is a best practice.
- Avoid disorienting users by causing unexplained application blocks or critical errors that they cannot resolve themselves.
- Handling error through try-catch permits to control messages and can make difference for the UX.

Ca' Foscari University of Venice

ATM example – concurrency test

ConcurrencyTest.java

src > test > java > com > atm > ConcurrencyTest.java

```java
/**
 * Concurrency test class to demonstrate race conditions in multi-threaded banking operations.
 * This test simulates multiple users performing simultaneous transactions on the same account
 * to expose potential thread safety issues.
 *
 * The test is designed to fail when the BankAccount class has race conditions
 * and pass when proper synchronization is implemented.
 *
 * @author ATM Simulator
 * @version 1.0
 * @since 2025-10-21
 */
public class ConcurrencyTest {

    private static final int NUM_THREADS = 20;
    private static final int OPERATIONS_PER_THREAD = 100;
    private static final double INITIAL_BALANCE = 10000.0;
    private static final double DEPOSIT_AMOUNT = 10.0;
    private static final double WITHDRAW_AMOUNT = 5.0;

    private static int testsRun = 0;
    private static int testsPassed = 0;
    private static boolean verbose = true;

    /**
     * Main method to run all concurrency tests
```

- Can your application have concurrency issues?
- The correct use of programming languages (or the choice of language itself) can help prevent this kind of issue.
- E.g., node.js is single thread language: you will never be in a situation where two lines of JavaScript are executed at the same time.

ATM example – concurrency test

```
🧪 Running Concurrency Test...
==============================

=== ATM Concurrency Test Suite ===
Testing thread safety with 20 concurrent users
Each user performs 100 operations
Initial account balance: $10000.0
================================================

🧪 Test 1: Concurrent Deposits and Withdrawals
------------------------------------------------
Starting all threads simultaneously...
Thread 13 starting operations...
Thread 11 starting operations...
Deposited: $10.00. New balance: $10020.00
Thread 14 starting operations...
Deposited: $10.00. New balance: $10030.00
Thread 3 starting operations...
Deposited: $10.00. New balance: $10040.00
Thread 10 starting operations...
Deposited: $10.00. New balance: $10050.00
Deposited: $10.00. New balance: $10010.00
Thread 4 starting operations...
Deposited: $10.00. New balance: $10060.00
Thread 1 starting operations...
Thread 19 starting operations...
Deposited: $10.00. New balance: $10070.00
Thread 18 starting operations...
Deposited: $10.00. New balance: $10090.00
```

```
Withdrawn: $5.00. New balance: $14990.00
Withdrawn: $5.00. New balance: $14985.00
Thread 5 completed all operations
Withdrawn: $5.00. New balance: $14980.00
Withdrawn: $5.00. New balance: $14975.00
Thread 0 completed all operations
Withdrawn: $5.00. New balance: $14970.00
Thread 11 completed all operations
Deposited: $10.00. New balance: $14980.00
Thread 15 completed all operations
Deposited: $10.00. New balance: $14990.00
Thread 1 completed all operations
Withdrawn: $5.00. New balance: $14985.00
Withdrawn: $5.00. New balance: $14980.00
Thread 13 completed all operations
Thread 4 completed all operations

📊 Results:
Successful deposits: 1000
Successful withdrawals: 1000
Failed operations: 0
Expected balance: $15000.00
Actual balance: $14980.00
Balance difference: $20.00
❌ Balance inconsistency detected - RACE CONDITION FOUND!
This indicates a thread safety issue in the BankAccount implementation
```

Ca' Foscari University of Venice

Deliverables

- Tests book (or test cases)
  - track of all tests (each test should be described clearly and define properly the expected results)
  - track of status of each test (passed, failed, ...)
- Report about execution of tests
- Before to perform them, it is important to define metrics and goals to evaluate correctly the obtained results

Individual Test Results

Test Name	Description	Status	Expected Result	Actual Result	Notes
`testConstructorWithZeroBalance`	Verify default constructor creates account with $0 balance	✅ **PASSED**	Balance = $0.00	Balance = $0.00	Constructor working correctly
`testConstructorWithInitialBalance`	Verify parameterized constructor with positive balance	✅ **PASSED**	Balance = Initial Amount	Balance = Initial Amount	Initialization successful
`testConstructorWithNegativeBalance`	Verify exception thrown for negative initial balance	✅ **PASSED**	IllegalArgumentException	IllegalArgumentException	Validation working
`testGetBalance`	Verify balance retrieval functionality	✅ **PASSED**	Current balance returned	Current balance returned	Getter method functional
`testValidDeposit`	Test successful deposit operation	✅ **PASSED**	Balance increased by deposit	Balance increased correctly	Deposit logic correct
`testInvalidDepositZero`	Test deposit with zero amount	✅ **PASSED**	IllegalArgumentException	IllegalArgumentException	Zero validation working
`testInvalidDepositNegative`	Test deposit with negative amount	✅ **PASSED**	IllegalArgumentException	IllegalArgumentException	Negative validation working
`testValidWithdrawal`	Test successful withdrawal operation	✅ **PASSED**	Balance decreased by withdrawal	Balance decreased correctly	Withdrawal logic correct
`testInvalidWithdrawalZero`	Test withdrawal with zero amount	✅ **PASSED**	IllegalArgumentException	IllegalArgumentException	Zero validation working
`testInvalidWithdrawalNegative`	Test withdrawal with negative amount	✅ **PASSED**	IllegalArgumentException	IllegalArgumentException	Negative validation working
`testInsufficientFunds`	Test withdrawal exceeding balance	✅ **PASSED**	InsufficientFundsException	InsufficientFundsException	Overdraft protection working
`testMultipleOperations`	Test sequence of deposits and withdrawals	✅ **PASSED**	Correct final balance	Correct final balance	Sequential operations work
`testThreadSafety`	Basic thread safety test with concurrent operations	✅ **PASSED**	Expected balance: $1,500.00	Actual balance: $1,500.00	Thread safety confirmed
`testToString`	Test string representation of account	✅ **PASSED**	Formatted balance string	Formatted balance string	toString() working
`testEquals`	Test equality comparison between accounts	✅ **PASSED**	Proper equality logic	Proper equality logic	equals() implemented correctly
`testHashCode`	Test hash code generation consistency	✅ **PASSED**	Consistent hash codes	Consistent hash codes	hashCode() working

Ca' Foscari University of Venice

Hint

- If the test has the role to validate what you have done: work within the customer (or who required your software) and obtain its approval about
  - test cases
  - metrics for evaluation
  - acceptable threshold

SFCC | Status - Priority Go Live 1

Stato	High	Medium	Low	Totale(x)
IN CORSO	8	0	0	8
BACKLOG	3	1	1	5
IN UAT	15	3	2	20
UAT KO	1	0	0	1
ANALYSIS IN PROGRESS	4	0	0	4
TECH ANALYSIS IN PR...	1	0	0	1
IN SIT	2	1	0	3
Ticket univoci totali:	**34**	**5**	**3**	**42**

Raggruppate per: Priorità

Visualizzazione di **7** statistiche su **7**.

Unit tests

- A unit test is a type of software testing where **individual units** or components of a program are tested in isolation. The purpose is to **ensure that each part of the software functions as expected**.
- Unit tests help to identify bugs early in the development cycle and improve code reliability by ensuring that each part of the code behaves correctly under different conditions.

- Key features
  - Isolated: Unit tests focus on a single "unit" of code, typically a function or method, without depending on other parts of the program.
  - Repeatable: Unit tests should give the same result every time they run.
  - Automated: They are often run automatically as part of a continuous integration (CI) pipeline to ensure code quality.

Ca' Foscari University of Venice

Unit tests

BankAccountJUnitTest.java U ×

src > test > java > com > atm > BankAccountJUnitTest.java

```java
/**
 */
@DisplayName("BankAccount JUnit Test Suite")
public class BankAccountJUnitTest {

    private BankAccount account;

    @BeforeEach
    @DisplayName("Initialize test account")
    void setUp() {
        account = new BankAccount();
    }

    @Nested
    @DisplayName("Constructor Tests")
    class ConstructorTests {

        @Test
        @DisplayName("Default constructor should create account with zero balance")
        void testDefaultConstructor() {
            BankAccount newAccount = new BankAccount();
            assertEquals(0.0, newAccount.getBalance(), 0.001,
                "Default constructor should initialize balance to 0.0");
        }

        @Test
        @DisplayName("Constructor with positive initial balance")
        void testConstructorWithPositiveBalance() {
            double initialBalance = 100.50;
            BankAccount newAccount = new BankAccount(initialBalance);
            assertEquals(initialBalance, newAccount.getBalance(), 0.001,
                "Constructor should set initial balance correctly");
        }

        @Test
        @DisplayName("Constructor with zero initial balance")
        void testConstructorWithZeroBalance() {
            BankAccount newAccount = new BankAccount(0.0);
            assertEquals(0.0, newAccount.getBalance(), 0.001,
                "Constructor should accept zero as valid initial balance");
        }

        @Test
        @DisplayName("Constructor should reject negative initial balance")
        void testConstructorWithNegativeBalance() {
            IllegalArgumentException exception = assertThrows(
                IllegalArgumentException.class,
```

```java
    @Nested
    @DisplayName("Balance Operations")
    class BalanceTests {

        @Test
        @DisplayName("getBalance should return current balance")
        void testGetBalance() {
            assertEquals(0.0, account.getBalance(), 0.001,
                "Initial balance should be 0.0");
        }

        @Test
        @DisplayName("Balance should be consistent after operations")
        void testBalanceConsistency() {
            // Perform multiple operations
            account.deposit(100.0);
            assertEquals(100.0, account.getBalance(), 0.001);

            account.withdraw(30.0);
            assertEquals(70.0, account.getBalance(), 0.001);

            account.deposit(25.5);
            assertEquals(95.5, account.getBalance(), 0.001);
        }
    }

    @Nested
    @DisplayName("Deposit Operations")
    class DepositTests {

        @Test
        @DisplayName("Valid deposit should increase balance")
        void testValidDeposit() {
            double initialBalance = account.getBalance();
            double depositAmount = 50.0;

            account.deposit(depositAmount);

            assertEquals(initialBalance + depositAmount, account.getBalance(), 0.001,
                "Balance should increase by deposit amount");
        }

        @Test
        @DisplayName("Multiple deposits should accumulate correctly")
        void testMultipleDeposits() {
```

Ca' Foscari University of Venice

Unit tests - execution

```
JUnit Jupiter ✔
└─ BankAccount JUnit Test Suite ✔
   ├─ Error Message Validation ✔
   │  ├─ Negative deposit error message should be descriptive ✔
   │  └─ Insufficient funds error message should be descriptive ✔
   ├─ Edge Cases and Boundary Conditions ✔
   │  ├─ Very small amounts should be handled correctly ✔
   │  ├─ Large amounts should be handled correctly ✔
   │  ├─ Precision with decimal amounts ✔
   │  └─ Sequential operations should maintain consistency ✔
   ├─ Thread Safety Tests ✔
   │  ├─ Repeated thread safety stress test ✔
   │  │  ├─ Thread safety stress test - iteration 1 ✔
   │  │  ├─ Thread safety stress test - iteration 2 ✔
   │  │  ├─ Thread safety stress test - iteration 3 ✔
   │  │  ├─ Thread safety stress test - iteration 4 ✔
   │  │  └─ Thread safety stress test - iteration 5 ✔
   │  ├─ Concurrent mixed operations should be thread-safe ✔
   │  └─ Concurrent deposits should be thread-safe ✔
   ├─ Object Methods ✔
   │  ├─ toString should return formatted balance ✔
   │  ├─ hashCode should be consistent ✔
   │  ├─ equals should work correctly for same balance ✔
   │  ├─ equals should return false for different balances ✔
   │  └─ equals should handle null correctly ✔
   ├─ Withdrawal Operations ✔
   │  ├─ Withdrawal of entire balance should leave zero ✔
   │  ├─ Withdrawal exceeding balance should throw InsufficientFundsException ✔
   │  ├─ Valid withdrawal amounts should work correctly ✔
   │  │  ├─ [1] 10.0 ✔
   │  │  ├─ [2] 25.5 ✔
   │  │  ├─ [3] 50.0 ✔
   │  │  └─ [4] 99.99 ✔
   │  ├─ Valid withdrawal should decrease balance ✔
   │  ├─ Multiple withdrawals should accumulate correctly ✔
   │  ├─ Withdrawal with negative amount should throw exception ✔
   │  └─ Withdrawal with zero amount should throw exception ✔
   ├─ Deposit Operations ✔
```

Integration test

- An integration test is a type of software testing where individual units or components are combined and tested as a group to evaluate how they interact with each other.
- The goal of integration testing is to identify defects that occur when multiple units or systems are brought together and to ensure that the integrated system functions correctly as a whole.

- Key features
  - Tests interactions: It focuses on the communication between different modules or components of the application.
  - Ensures compatibility: It checks that individual units work together as expected after being integrated.
  - Builds on unit tests: Integration testing typically happens after unit testing and before system testing.
  - Uncovers interface issues: It helps detect issues related to interfaces, data flow, and shared dependencies.

Integration test - examples

2. Integration Tests (17 tests) - `src/integration-test/java/`

Multi-User Scenarios (2 tests)

- **testMultipleUsersAccessingSameAccount**: 10 users simultaneously access same account to test thread safety
- **testMixedOperationsFromMultipleThreads**: 8 threads execute mixed operations (deposits/withdrawals) simultaneously

System Integration Scenarios (2 tests)

- **testCompleteAtmTransactionWorkflow**: Complete ATM workflow with balance check → deposit → withdrawal → error handling
- **testAtmSimulatorWithMultipleAccounts**: System test with multiple simultaneous accounts to verify isolation

Performance and Load Testing (1 test)

- **testHighVolumeTransactionProcessing**: Processing 1000 concurrent transactions to test performance

Error Recovery and Edge Cases (2 tests)

- **testSystemBehaviorUnderExtremeConditions**: System behavior under extreme conditions (very large amounts)
- **testRecoveryFromExceptionScenarios**: Recovery from multiple exception scenarios

Advanced Integration Tests (10 tests)

- **testBankBranchSimulation**: Bank branch simulation with 3 ATMs and 15 clients
- **testScalabilityWithVaryingLoads**: Parameterized scalability test with varying loads (1, 5, 10, 20, 50 users)
- **testExtremeConcurrentAccessStressTest**: Extreme concurrent access stress test (100 threads, 50 operations each)
- **testMemoryAndResourceLeakDetection**: Memory leak detection with 1000 iterations
- **testEndToEndBankingWorkflowIntegration**: Complete end-to-end banking workflow
- **testMultiAccountTransferSimulation**: Multi-account transfer simulation

End2end test

- End-to-End (E2E) testing is a type of software testing that verifies the complete flow of an application from start to finish, simulating real user scenarios.
- The goal is to ensure that all components of the system, including external dependencies (such as databases, networks, and third-party services), work together as expected.
- E2E tests check the behavior of the entire application, from the user interface to the back-end, databases, and integrations, to identify any issues that may arise in real-world conditions.

- Key features
  - Complete system testing: Tests the entire application as a whole, not just isolated components.
  - Real-world scenarios: Simulates how users interact with the application.
  - Involves external systems: May include testing interactions with external APIs, databases, and other services.
  - User-focused: Verifies that the system works from the user's perspective, ensuring a smooth user experience.

End2end test

- More complex tests to perform
- Approaches:
  - Manual end-to-end tests (a lot of people, devices, tools, ...)
  - Automation of end-to-end tests (e.g., cucumber + cypress; a lot of effort to write them down)
  - Using external service (e.g., Tryber - https://www.tryber.me/, a crowdtesting community)

End2end tests - examples

3. End-to-End Tests (13 tests) - `src/e2e-test/java/`

Complete User Journey Tests (3 tests)

- **testCompleteAtmSession**: Complete ATM session from account creation to exit
- **testAtmSessionWithErrorRecovery**: Session with error handling and recovery
- **testMultipleAtmSessions**: Multiple simultaneous ATM sessions simulation

System Integration and Data Flow Tests (2 tests)

- **testDataPersistenceWithinSession**: Data persistence across operations within single session
- **testResourceManagementDuringSession**: Memory and resource management during intensive session

User Interface and Experience Tests (2 tests)

- **testMenuNavigationAndPrompts**: Menu navigation and user prompts
- **testErrorMessagesAndGuidance**: Error messages and user guidance

Performance and Reliability Tests (2 tests)

- **testStartupShutdownPerformance**: Application startup and shutdown performance
- **testRapidUserInteractions**: Rapid user interactions testing

E2E Comparison Tests (4 tests)

- **testCompleteUserExperienceWithUI**: Complete user experience with UI interaction
- **testComponentInteractionWithoutUI**: Component interaction without UI (comparison)
- **testUserMistakesAndRecovery**: User error handling and recovery
- **testStartupShutdownFlow**: Complete startup-shutdown application flow

Frontend test

- Main tools available on the market:
  - BrowserStack - a cloud-based platform that allows for cross-browser and cross-device testing of web and mobile applications. It provides automated and live interactive testing for websites across thousands of real devices and browsers.
  - Applitools - an AI-powered visual testing tool that automates the process of ensuring that an application's UI looks and functions correctly. It uses visual comparisons to detect differences in UI across different browsers, platforms, and screen sizes.
  - Sauce Labs - a cloud-based platform that provides automated cross-browser and cross-device testing. It allows users to test the UI/UX of applications on a wide range of devices and browsers, both real and virtual.
  - ... and many more

Ca' Foscari University of Venice

FE tests – UX of a CLI

Admin CLI Command Tests:

1. **testAdminHelpCommand** ✅
   - Verifies help command shows all expected admin commands
   - Checks for: list, create, balance, unlock, server, quit commands
   - **Result**: PASS - All commands properly documented
2. **testAdminListCommand** ✅
   - Tests account listing functionality with proper formatting
   - Verifies presence of test accounts (Mario Rossi, Luigi Verdi, Anna Bianchi)
   - **Result**: PASS - Accounts displayed with correct format
3. **testAdminCreateAccountCommand** ❌
   - Tests new account creation with balance initialization
   - **Issue**: Creation confirmation and user verification failed
   - **Recommendation**: Improve account creation feedback messages
4. **testAdminBalanceCommand** ❌
   - Tests balance display for specific accounts
   - **Issue**: Numeric balance format not properly detected
   - **Recommendation**: Standardize balance output format (e.g., €1,000.00)
5. **testAdminUnlockCommand** ✅
   - Tests account unlocking functionality
   - **Result**: PASS - Provides clear confirmation message
6. **testAdminInvalidCommand** ✅
   - Tests graceful handling of unknown commands
   - **Result**: PASS - Shows helpful error messages
7. **testAdminQuitCommand** ✅
   - Tests proper exit from admin CLI
   - **Result**: PASS - Shows goodbye message

📱 Client CLI Tests (AtmClientCliTest.java)

Test Results: 12/13 tests passed (92% success rate)

Client Command Tests:

1. **testClientHelpCommand** ✅
   - Verifies help shows all client commands
   - **Result**: PASS - All commands documented (accounts, balance, deposit, withdraw, health, quit)
2. **testClientAccountsCommand** ✅
   - Tests formatted account table display
   - **Result**: PASS - Shows tabular format with headers and status
3. **testClientBalanceCommand** ❌
   - Tests balance display formatting
   - **Issue**: Numeric amount format not properly detected
   - **Recommendation**: Ensure consistent €X,XXX.XX format
4. **testClientDepositCommand** ✅
   - Tests deposit operation feedback
   - **Result**: PASS - Clear success confirmation with new balance
5. **testClientWithdrawCommand** ✅
   - Tests withdrawal operation feedback
   - **Result**: PASS - Clear success confirmation with balance update
6. **testClientHealthCommand** ✅
   - Tests server connectivity check
   - **Result**: PASS - Executes without errors
7. **testClientInvalidCommand** ✅
   - Tests unknown command handling
   - **Result**: PASS - Helpful error messages with guidance
8. **testClientQuitCommand** ✅
   - Tests polite exit from client
   - **Result**: PASS - Shows thank you message

User Experience Tests:

9. **testInteractiveMode** ✅
   - Tests interactive prompts for missing parameters
   - **Result**: PASS - Prompts for account number when missing
10. **testParameterModeVsInteractive** ✅
    - Tests consistency between parameter and interactive modes
    - **Result**: PASS - Both modes produce same results
11. **testServerConnectionError** ✅
    - Tests handling of server connection failures
    - **Result**: PASS - Clear error messages about connection issues
12. **testInvalidAccountHandling** ✅
    - Tests error handling for non-existent accounts
    - **Result**: PASS - Clear "not found" error messages
13. **testInvalidAmountHandling** ✅
    - Tests validation of negative amounts
    - **Result**: PASS - Appropriate error messages for invalid amounts

Security test

- In order to test security, we have two kind or tests:
  - Vulnerability Assessment (VA)
  - Penetration Test (PT)
- You have to consider many aspects and to perform this kind of tests is it important have all the information about architecture, environments, application stack, ...
  - you are not able to know which of these information the attacker could have before its attack
- Generally, it's a third party that perform the test in sake of reliability

Ca' Foscari University of Venice

Load test

```
2   <jmeterTestPlan version="1.2" properties="5.0" jmeter="5.5">
3     <hashTree>
14      <hashTree>
28        <hashTree>
38          <HTTPSamplerProxy guiclass="HttpTestSampleGui" testclass="HTTPSamplerProxy" testname="HTTP Request" enabled="true">
41            <collectionProp name="Arguments.arguments">
42              <elementProp name="" elementType="HTTPArgument">
43                <boolProp name="HTTPArgument.always_encode">false</boolProp>
44                <stringProp name="Argument.value">{&#xd;
45  &quot;wechatid&quot;: &quot;o1RiMwm5m26KhpkS5uRmmu8bysz4&quot;&#xd;
46  }</stringProp>
47                <stringProp name="Argument.metadata">=</stringProp>
48              </elementProp>
49            </collectionProp>
50          </elementProp>
51          <stringProp name="HTTPSampler.domain">cdh-qas-scc.[redacted]-cdh-qas.svc.cluster.local</stringProp>
52          <stringProp name="HTTPSampler.port">5000</stringProp>
53          <stringProp name="HTTPSampler.protocol">http</stringProp>
54          <stringProp name="HTTPSampler.contentEncoding">></stringProp>
55          <stringProp name="HTTPSampler.path">cdh-scc/Account/P011/S004/SearchByWeChatId</stringProp>
56          <stringProp name="HTTPSampler.method">POST</stringProp>
57          <boolProp name="HTTPSampler.follow_redirects">true</boolProp>
58          <boolProp name="HTTPSampler.auto_redirects">false</boolProp>
59          <boolProp name="HTTPSampler.use_keepalive">true</boolProp>
60          <boolProp name="HTTPSampler.DO_MULTIPART_POST">false</boolProp>
61          <stringProp name="HTTPSampler.embedded_url_re">></stringProp>
62          <stringProp name="HTTPSampler.connect_timeout">></stringProp>
63          <stringProp name="HTTPSampler.response_timeout">></stringProp>
64        </HTTPSamplerProxy>
65        <hashTree/>
66        <ResultCollector guiclass="ViewResultsFullVisualizer" testclass="ResultCollector" testname="View Results Tree" enabled="true">
67          <boolProp name="ResultCollector.error_logging">false</boolProp>
68          <objProp>
69            <name>saveConfig</name>
70            <value class="SampleSaveConfiguration">
71              <time>true</time>
72              <latency>true</latency>
73              <timestamp>true</timestamp>
74              <success>true</success>
```

Ca' Foscari University of Venice

Load test

```
summary =  46852 in 00:02:57 =  264.9/s Avg: 15894 Min:    19 Max: 127959 Err:  8867 (18.93%)
summary +   7790 in 00:00:30 =  259.4/s Avg: 21186 Min:  2995 Max: 127328 Err:  1613 (20.71%) Active: 5000 Started: 5000 Finished: 0
summary =  54642 in 00:03:27 =  264.1/s Avg: 16649 Min:    19 Max: 127959 Err: 10480 (19.18%)
summary +   7162 in 00:00:30 =  239.2/s Avg: 13545 Min:  1926 Max: 127340 Err:   883 (12.33%) Active: 5000 Started: 5000 Finished: 0
summary =  61804 in 00:03:57 =  260.9/s Avg: 16289 Min:    19 Max: 127959 Err: 11363 (18.39%)
summary +   7403 in 00:00:30 =  245.5/s Avg: 10374 Min:   283 Max: 93248 Err:   952 (12.86%) Active: 4999 Started: 5000 Finished: 1
summary =  69207 in 00:04:27 =  259.2/s Avg: 15656 Min:    19 Max: 127959 Err: 12315 (17.79%)
summary +  12001 in 00:00:30 =  401.6/s Avg: 24251 Min:    29 Max: 127533 Err:  2920 (24.33%) Active: 4989 Started: 5000 Finished: 11
summary =  81208 in 00:04:57 =  273.5/s Avg: 16927 Min:    19 Max: 127959 Err: 15235 (18.76%)
summary +  12866 in 00:00:30 =  429.4/s Avg: 11894 Min:   178 Max: 127482 Err:  1188 (9.23%) Active: 4972 Started: 5000 Finished: 28
summary =  94074 in 00:05:27 =  287.8/s Avg: 16238 Min:    19 Max: 127959 Err: 16423 (17.46%)
summary +  19929 in 00:00:30 =  663.1/s Avg:  9016 Min:   111 Max: 127504 Err:   819 (4.11%) Active: 4939 Started: 5000 Finished: 61
summary = 114003 in 00:05:57 =  319.4/s Avg: 14976 Min:    19 Max: 127959 Err: 17242 (15.12%)
summary +  19678 in 00:00:30 =  656.7/s Avg:  7653 Min:    78 Max: 127500 Err:   894 (4.54%) Active: 4879 Started: 5000 Finished: 121
summary = 133681 in 00:06:27 =  345.5/s Avg: 13898 Min:    19 Max: 127959 Err: 18136 (13.57%)
summary +  24562 in 00:00:30 =  819.1/s Avg:  6882 Min:    24 Max: 127450 Err:   526 (2.14%) Active: 4753 Started: 5000 Finished: 247
summary = 158243 in 00:06:57 =  379.6/s Avg: 12809 Min:    19 Max: 127959 Err: 18662 (11.79%)
summary +  26000 in 00:00:30 =  865.7/s Avg:  6046 Min:   233 Max: 127686 Err:   176 (0.68%) Active: 4570 Started: 5000 Finished: 430
summary = 184243 in 00:07:27 =  412.3/s Avg: 11854 Min:    19 Max: 127959 Err: 18838 (10.22%)
summary +  25927 in 00:00:30 =  864.1/s Avg:  5162 Min:    72 Max: 127422 Err:   137 (0.53%) Active: 4234 Started: 5000 Finished: 766
summary = 210170 in 00:07:57 =  440.7/s Avg: 11029 Min:    19 Max: 127959 Err: 18975 (9.03%)
summary +  25200 in 00:00:30 =  840.9/s Avg:  4934 Min:    30 Max: 93653 Err:   143 (0.57%) Active: 3732 Started: 5000 Finished: 1268
summary = 235370 in 00:08:27 =  464.4/s Avg: 10376 Min:    19 Max: 127959 Err: 19118 (8.12%)
summary +  26670 in 00:00:30 =  889.1/s Avg:  4153 Min:    26 Max: 127439 Err:   205 (0.77%) Active: 2747 Started: 5000 Finished: 2253
summary = 262040 in 00:08:57 =  488.1/s Avg:  9743 Min:    19 Max: 127959 Err: 19323 (7.37%)
summary +  25672 in 00:00:30 =  855.8/s Avg:  2497 Min:    12 Max: 31473 Err:     0 (0.00%) Active: 1161 Started: 5000 Finished: 3839
summary = 287712 in 00:09:27 =  507.5/s Avg:  9096 Min:    12 Max: 127959 Err: 19323 (6.72%)
summary +  11631 in 00:00:30 =  387.1/s Avg:  1506 Min:    12 Max:  7487 Err:     0 (0.00%) Active: 102 Started: 5000 Finished: 4898
summary = 299343 in 00:09:57 =  501.5/s Avg:  8802 Min:    12 Max: 127959 Err: 19323 (6.46%)
summary +    657 in 00:00:05 =  122.6/s Avg:   397 Min:    12 Max:  1317 Err:     0 (0.00%) Active: 0 Started: 5000 Finished: 5000
summary = 300000 in 00:10:02 =  498.1/s Avg:  8783 Min:    12 Max: 127959 Err: 19323 (6.44%)
Tidying up ...    @ July 7, 2023 2:18:05 PM UTC (1688739485410)
... end of run
```

```
summary +   3994 in 00:00:06 =  716.7/s Avg:  1415 Min:    16 Max:  3828 Err:     0 (0.00%) Active: 5000 Started: 5000 Finished: 0
summary +  44126 in 00:00:30 = 1470.1/s Avg:  3131 Min:    82 Max: 30343 Err:   333 (0.75%) Active: 5000 Started: 5000 Finished: 0
summary =  48120 in 00:00:36 = 1352.1/s Avg:  2989 Min:    16 Max: 30343 Err:   333 (0.69%)
summary +  47370 in 00:00:30 = 1577.5/s Avg:  3245 Min:   269 Max: 30429 Err:   885 (1.87%) Active: 5000 Started: 5000 Finished: 0
summary =  95490 in 00:01:06 = 1455.3/s Avg:  3116 Min:    16 Max: 30429 Err:  1218 (1.28%)
summary +  47526 in 00:00:30 = 1586.1/s Avg:  3190 Min:    45 Max: 30334 Err:   519 (1.09%) Active: 5000 Started: 5000 Finished: 0
summary = 143016 in 00:01:36 = 1496.2/s Avg:  3140 Min:    16 Max: 30429 Err:  1737 (1.21%)
summary +  47534 in 00:00:30 = 1580.3/s Avg:  3192 Min:   177 Max: 30457 Err:   414 (0.87%) Active: 4969 Started: 5000 Finished: 31
summary = 190550 in 00:02:06 = 1516.4/s Avg:  3153 Min:    16 Max: 30457 Err:  2151 (1.13%)
summary +  47061 in 00:00:30 = 1572.7/s Avg:  3105 Min:   244 Max: 30292 Err:   340 (0.72%) Active: 4482 Started: 5000 Finished: 518
summary = 237611 in 00:02:36 = 1527.2/s Avg:  3144 Min:    16 Max: 30457 Err:  2491 (1.05%)
summary +  47155 in 00:00:30 = 1569.2/s Avg:  2297 Min:    14 Max: 30288 Err:   299 (0.63%) Active: 1878 Started: 5000 Finished: 3122
summary = 284766 in 00:03:06 = 1534.0/s Avg:  3003 Min:    14 Max: 30457 Err:  2790 (0.98%)
summary +  15234 in 00:00:22 =  699.4/s Avg:  1243 Min:    12 Max: 30035 Err:   121 (0.79%) Active: 0 Started: 5000 Finished: 5000
summary = 300000 in 00:03:27 = 1446.3/s Avg:  2914 Min:    12 Max: 30457 Err:  2911 (0.97%)
Tidying up ...    @ July 7, 2023 5:04:51 PM UTC (1688749491847)
... end of run
```